

InterOpera — DevOps / MLOps Engineer Take-Home

Issued by: InterOpera Platform Engineering **Estimated effort:** 22–28 hours over 7 days **Submission deadline:** 7 days from receipt

We are not looking for a polished platform — we are looking for the engineer who can take a service the application team built, put it into production **as code**, write the retrieval backend that grounds its answers, and be trusted with the pager: gated model rollouts, alerts that actually fire, and incidents root-caused from telemetry.

Background

Our enterprise clients are regulated financial firms. One of them — **Meridian Asset Management**, a fictional SGD-denominated, MAS-supervised asset manager — requires that the language model answering questions about its fund documents runs **inside its own tenant**: self-hosted, no data leaving the environment, fully observable, and auditable. No frontier API can be in the request path.

The application team has built the two services you will operate:

- **meridian-slm** — a small specialized LLM served behind an OpenAI-compatible API. For this assignment it is a **mock inference server** that simulates the real thing (deterministic answers, realistic latency, Prometheus metrics including simulated GPU/KV-cache gauges) so that **no GPU and no cloud account are needed**. Treat it exactly as you would a vLLM server loading versioned model weights on a GPU node.
- **gateway** — the client-facing service that fronts the model server.

There is one piece nobody has built yet, and it is yours. The model was fine-tuned on Meridian's documents, but fine-tuned knowledge goes stale and cannot be cited — and Meridian's auditors require that chat answers be **grounded in the current document corpus, with the supporting passages cited**. The model server already supports grounded answering (it answers only from context you provide, and refuses otherwise — see the starter README for the protocol). **You will write and operate the retrieval service (RAG) that feeds it:** ingest the corpus into a vector store, retrieve, and ground.

The ML team ships new model builds (**1.1** , a quantized rebuild; **2.0** , a new fine-tune). Meridian's service agreement is blunt:

Meridian service agreement (the numbers you operate against):

- **Availability:** 99.5% monthly on the completion endpoint.
- **Latency:** $p_{95} \leq 1200$ ms under the reference load (8 concurrent clients).
- **Quality:** $\geq 90\%$ accuracy on the acceptance set (`eval/eval_set.jsonl`) for any model version serving live traffic — **on the direct path and on the RAG path**.
- **Grounding:** RAG answers must cite the corpus passages they came from; a question the corpus cannot answer must be refused, not guessed.

Your task: build and operate the production platform for this stack — Kubernetes, IaC, the RAG backend and its vector store, CI/CD with a model-quality gate, observability — and handle one production incident. You own the platform and the retrieval data plane; you do not own the model weights or the gateway's application code.

The hard constraints

These are non-negotiable. Everything we evaluate traces back to them.

1. **Reproducible bring-up.** From a clean machine with only your documented prerequisites installed (e.g., Docker, kind/k3d/minikube, kubectl, Terraform), **one documented command** brings up the entire platform — including the vector store and your RAG service. We will destroy everything and re-create it.
2. **Everything as code.** Every cluster object, pipeline step, dashboard, and alert rule is declared in the repository. If reproducing your result requires a manual step — `kubectl edit`, clicking in a UI — the constraint is broken.
3. **Gated model delivery.** A model version reaches live traffic only through a progressive rollout (canary or blue/green — your call, defend it) that is **promoted or rolled back automatically by your evaluation gate** — no human in the loop, and the decision evidence is recorded. Rolling out `meridian-slm:2.0` must end with 2.0 **rolled back automatically**; rolling out `1.1` must end with 1.1 **promoted**.
4. **Provable observability.** You define SLOs from the Meridian agreement; your alerts must **demonstrably fire**. Using the provided fault injection, we will break the model server and wait for your alert.
5. **Self-hosted, no egress.** At runtime the request path works with no external services — no frontier LLM APIs, no hosted embedding APIs, no managed cloud services. (This is the data-residency requirement in miniature. Downloading public images, charts, and open-source embedding models during setup is fine.)

How you satisfy them — which local Kubernetes, which vector store, which rollout mechanism, which observability stack, how the gate is wired — is yours to derive and defend.

Materials provided

Find them in `starter/` (see its README for details, including the grounded request protocol):

ITEM	WHAT IT IS
<code>services/model-server/</code>	The mock <code>meridian-slm</code> inference server. <code>MODEL_VERSION</code> (<code>1.0</code> , <code>1.1</code> , <code>2.0</code>) selects the model build. Supports direct and grounded (RAG) answering. Do not modify its answer or latency logic — it stands in for model weights. Exposes <code>/metrics</code> and a fault-injection endpoint (<code>/admin/fault</code>) for your alerting demo.

ITEM	WHAT IT IS
services/gateway/	The client-facing gateway, written by the application team.
corpus/	The Meridian fund document corpus — the only knowledge source for your RAG service. It covers more than one Meridian fund .
eval/eval_set.jsonl	Meridian's acceptance set. Prompts are sent verbatim; the model is deterministic, so a response is correct iff it exactly matches expected . Used for both the direct and the RAG path.
loadgen/loadgen.py	Load generator. The reference load is <code>--concurrency 8 --duration 60</code> with the acceptance-set prompts.
docker-compose.yml	A local starting point so you can see the stack behave. Not the deliverable.

What we evaluate (in priority order)

1. **Do the five constraints hold?** We will, concretely: destroy and re-create the platform from your README on a clean machine; run your rollout of [1.1](#) (expect automatic promotion) and of [2.0](#) (expect automatic rollback) without touching anything; inject the fault and wait for your alert to fire; run the reference load before and after your incident fix.
2. **Is the RAG service real?** We will ask it acceptance-set questions and spot-check that the citations point at the right corpus passages; ask it a question about the *other* Meridian fund; and ask it something the corpus cannot answer (expect an honest refusal, not a guess). Retrieval must actually go through your vector store.
3. **Is the gate real?** A gate must evaluate the **deployed canary over the network** and enforce its thresholds automatically. An eval script that a human reads and acts on is not a gate. A gate that never sees live canary traffic is not a gate.
4. **Is observability real?** Dashboards nobody alerts on are decoration. We care that the SLOs are derived from the client agreement and that the alert actually pages.
5. **Ops judgment.** The rollout strategy you chose and why; the quality of your post-mortem; whether your incident fix is minimal and evidence-driven; your GPU-readiness thinking in the memo.

Deliverables

100 points across six phases.

Phase 1 — Platform design memo (15 pts)

[docs/01_platform_memo.md](#) — prose for an engineer joining the on-call rotation, not marketing. Cover:

- **Architecture** — a diagram of your platform (cluster, delivery pipeline, RAG data plane, observability, traffic flow), and the trust boundary: what is client-facing, what is internal-only (the fault-injection endpoint must not be publicly exposed).

- **SLOs** — the SLIs you chose, targets derived from the Meridian agreement, and your alerting policy (what pages, what merely warns, and why).
- **Rollout strategy** — canary vs. blue/green for model versions: the one you chose, the tradeoff, and what "healthy enough to promote" means here.
- **RAG data plane** — your vector store choice (Qdrant, pgvector, Weaviate, Chroma, Milvus, ...) and why; the embedding pipeline and its lifecycle (what happens when the corpus changes — re-ingestion, index versioning); how this data plane scales when the corpus is 10,000 documents instead of three.
- **GPU-readiness** — today the model server is a mock. Write the plan for the real thing: what changes when `meridian-slm` is a vLLM server on GPU nodes. Node pools and device plugins; MIG vs. time-slicing; what signal you'd autoscale on (and why HPA-on-CPU is wrong for inference); KV-cache and batching implications for your latency SLO; quantization as a cost/latency lever; roughly how you'd model cost per 1M tokens. This section is where your MLOps depth shows — be specific, not aspirational.
- **Security & residency notes** — secrets handling, what egress the platform needs and when, and what you would add for a real regulated deployment.

Phase 2 — Platform as code (15 pts)

- The stack runs on a **local Kubernetes cluster** (kind, k3d, or minikube).
- **Terraform provisions the cluster and/or its add-ons**; Kubernetes objects are declared via manifests, Helm, or Kustomize. If you substitute another IaC tool, justify it in the memo.
- **One documented command** (`make up` or equivalent) from a clean machine to a fully running platform (constraint 1). `make down` destroys it.
- Production basics: pinned images, resource requests/limits, liveness and readiness probes, non-root containers, no secrets in git.

Phase 3 — RAG chat service (20 pts) ★

The backend you write. A small retrieval service (`services/rag-api/`, language your choice) that makes the chat agent **document-grounded**:

- **Ingestion**: chunk and embed the `corpus/` documents into a **real vector store running on your cluster**, deployed as code like everything else. Embeddings come from a **local model** (e.g., a sentence-transformers checkpoint pulled at build/setup time) — no hosted embedding APIs at runtime (constraint 5). Chunking strategy is your call; it is judged by outcome.
- **Query path**: `POST /v1/rag/chat` with `{"question": ...}` → embed the question → retrieve top-k from the vector store → assemble a grounded prompt per the starter protocol → call the model server → return the answer **plus citations** (source file and the retrieved passage(s)). Mind the model's 4,000-character context window.
- **Quality bar**: ≥ 90% on the acceptance set through this endpoint (Meridian agreement). The corpus covers more than one fund — retrieval that surfaces the wrong fund's passages will fail it.
- **Honesty**: when the model refuses (answer not in the retrieved context), your API surfaces the refusal and cites nothing — it must never dress a refusal up as an answer.
- **The rules**: your service's only knowledge source is `corpus/`. Reading `answers.json`, the model-server code, or the eval set's `expected` fields from the service is out of bounds — those stand in for model weights and the client's acceptance criteria.

Phase 4 — Gated model delivery (25 pts) ★

The core delivery loop.

- **Model versions are immutable, versioned artifacts** (e.g., image tags built from the starter with the version baked in) — what was deployed, when, and what the gate decided must be recorded and reviewable.
- A **rollout command/pipeline** (e.g., `make rollout VERSION=1.1`) that: deploys the version as a **canary** (or blue/green pair), runs your **evaluation harness against the deployed candidate over the network** — accuracy and latency on the **direct path**, and grounded accuracy on the **RAG path** (a model that regresses on faithfulness must be caught before it serves live traffic; how you point the RAG path at the canary during evaluation is your design call — document it) — compares the results to the thresholds you derived from the Meridian agreement, and then **promotes or rolls back automatically**, writing the decision and evidence to a file or log we can read afterwards.
- `1.1` must end up serving all traffic; `2.0` must never take over live traffic and must end rolled back — **with zero human intervention in either case** (constraint 3).
- A CI definition (GitHub Actions or similar) is welcome, but the rollout path must also be **runnable locally by us with one command** — we grade by running it, not by reading YAML.

Phase 5 — Observability & alerting (15 pts)

- A metrics stack (Prometheus + Grafana + Alertmanager, or equivalent), deployed **as code** — dashboards and alert rules live in the repo (constraint 2).
- Dashboards covering: rate/errors/duration for the gateway, the model server, **and your RAG service**; retrieval latency; tokens per second and context-token volume (on the RAG path, context dominates token cost); queue depth and in-flight requests; and the simulated GPU/KV-cache gauges — the panels an on-call engineer needs at 3 a.m.
- **SLO-based alerts** for availability and latency, derived from your Phase 1 SLOs (burn-rate or threshold — justify your choice).
- **Prove the alert fires** (constraint 4): a documented command (e.g., `make inject-fault`) that uses the model server's `/admin/fault` endpoint to raise the error rate; your alert must fire, and resolve after `make clear-fault`. Capture the evidence (Alertmanager state, screenshot) in `evidence/`.

Phase 6 — Production incident (10 pts)

Meridian went live and the pager went off:

Under the reference load (8 concurrent clients), the completion endpoint's p95 breaches the 1200 ms SLO by a wide margin — yet the model server looks healthy and nearly idle.

- **Reproduce it** with the provided load generator, against your deployed stack.
- **Root-cause it from your telemetry.** Your post-mortem must show the evidence chain — which graphs/metrics you looked at, what they showed, and how they narrowed the cause. A correct diagnosis with no telemetry evidence ("we read the code") scores poorly: we are testing whether your observability, not your code-reading luck, finds problems.

- **Fix it minimally.** You may change the gateway's code for this phase only, and the diff should be small enough to defend line by line.
- `docs/02_postmortem.md` — impact, timeline, root cause, the evidence chain, the fix, and prevention/detection (what alert or test catches the regression next time).
- **Before/after** reference-load results in `evidence/`.

Bonus (+5, capped; only if Phases 2–5 are complete)

- **Autoscaling on the right signal:** HPA (or KEDA) on a model-server metric (queue depth / in-flight requests), with a load-test demonstration (+2–3).
- **Hybrid retrieval or a reranker**, with a measured grounded-accuracy or latency delta against your baseline retriever (+1–2).
- **Traces:** OpenTelemetry across gateway → model server and RAG service → vector store → model server (+1–2).
- **GPU cost model:** a worked capacity/cost-per-1M-tokens plan for serving this workload on real GPUs, with actual numbers and assumptions (+1–2).
- **GPU scheduling demo:** simulate GPU scheduling on your local cluster (e.g., a fake device plugin advertising `nvidia.com/gpu`) with the model server requesting one (+2–3).

Submission format

A single repository:

```

/
├── README.md           ← prerequisites, the one bring-up command, and the
                        │   demo commands (rollout, fault injection, load test)
├── Makefile            ← or equivalent, wrapping every demo command
├── infra/              ← Terraform
├── deploy/             ← Kubernetes manifests / Helm / Kustomize
├── ci/                 ← pipeline definitions (if any)
├── observability/      ← dashboards + alert rules, as code
├── eval/               ← your evaluation harness (gate)
├── services/
│   ├── model-server/   ← provided (answer/latency logic unmodified)
│   ├── gateway/        ← provided (Phase 6 fix is the only allowed change)
│   └── rag-api/         ← the service you build (Phase 3)
├── corpus/             ← the provided documents
├── docs/
│   ├── 01_platform_memo.md
│   └── 02_postmortem.md
└── evidence/           ← gate decisions, alert firing, before/after loads,
                        │   RAG eval results

```

- **We must be able to run everything ourselves** on a machine with your documented prerequisites. `evidence/` supports your claims; it does not replace reproduction.
- **No LLM or embedding API keys are needed or allowed at runtime** (constraint 5). Using AI assistants to *build* your submission is fine — note briefly how you used them; you will defend every line in the follow-up interview.

- **Language:** all documentation and code comments in English.
-

Notes on scope

This is a one-week assignment, not a production migration. We expect a working platform that proves the constraints hold — not a hardened enterprise install.

- **Local is the target.** kind/k3d/minikube on a laptop. Do not build against a cloud account; do not use GPUs; do not replace the mock with a real model.
- **RAG is plumbing here, not IR research.** Sensible chunking, a stock local embedding model, and a properly operated vector store clear the quality bar. No rerankers or agent frameworks required — the bonus rewards them only with measured evidence.
- **Depth where it counts.** A boring, reliable stack with a real gate and a firing alert beats an exotic stack with either faked.
- **Error handling:** the happy path plus the failure modes this brief already exercises (bad model version, wrong-fund retrieval, refusals, injected faults, the incident). Nothing exhaustive.
- **Security:** the production basics listed in Phase 2; describe the rest in the memo instead of building it.
- **Assumptions are fine — state them** in the README or memo.

What we care about: does the platform rebuild from nothing on command; is the retrieval real and cited; is the gate real; do the alerts fire; and does your incident work show an engineer we would trust on call.

Questions about this assignment? Email recruiting@interopera.co. We respond within one business day.